

PyQCD 核心部分穷尽剖析：梯度流重整化胶子 TMD-PDF

opencode pure

2026-08-18

摘要

本报告对主仓库 PyQCD 的**核心部分**做穷尽剖析 (pure 技能), 旨在回答「这个仓库最强的是什么」。项目性质判定为**代码-物理交叉向** (胶子 TMD 计算代码与物理图像一一对应)。穷尽枚举 9 个核心候选, 分配三态: 深挖 C1 梯度流 RK3、C2 胶子 TMD 算符 (Clover+staple+O)、C3 Z_R 自重整化、C4 混合方案 λ 外推、C5 NLO 匹配核 $g_0..g_3$ 与 Z_{ij}、C7 torch 后端 +h5+MPI 并行; 浅析 C6 连续极限外推、C8 蒸馏管线、C9 数据分析链; 排除 cpp 占位与 refer 参考库。每深挖部分给出对象映射表 (代码符号 $\leftrightarrow$ 物理对象双向可查) 与独特性评估。结论: 最具竞争力的是「梯度流非微扰重整化 + 可乘性 O 组合 + Z_R/混合 + NLO 匹配核」的端到端自包含实现, 以及三后端可移植工程骨架。

目录

1 项目定位与核心部分清单 (穷尽枚举)	2
1.1 候选清单与三态分配	2
2 项目性质判定	2
3 核心部分深度剖析	3
3.1 C1 Wilson flow 梯度流 (RK3)	3
3.2 C2 胶子 TMD 算符 (Clover F + staple + O 组合)	3
3.3 C3 Z _R 自重整化	4
3.4 C4 混合方案 λ 外推	4
3.5 C5 NLO 匹配核 $g_0..g_3$ 与 Z _{ij}	5
3.6 C6 连续极限外推 (浅析)	5
3.7 C7 torch 后端 + h5 IO + MPI 并行 (深挖)	5
3.8 C8 蒸馏管线 run_pipeline (浅析)	6
3.9 C9 数据分析链 (浅析)	6
4 独特性评估	6
5 关键片段与公式	6
5.1 Wilson flow RK3 单步	6
5.2 Clover 场强 (节选)	6

5.3 NLO 匹配核 g_0..g_3 7

5.4 torch Tensor 补丁 8

6 参考源清单 9

7 结论与局限 9

1 项目定位与核心部分清单（穷尽枚举）

1.1 候选清单与三态分配

按仓库实测内容（代码/公式/映射占比）穷举核心候选，逐项分配三态（**深挖**/**浅析**/**排除**），证明「穷尽」而非「挑选」。

候选	类型	三态与理由
C1 Wilson flow 梯度流（RK3）	算法	深挖 : 非微扰重整化核心，Luescher 2010 RK3
C2 胶子 TMD 算符（Clover F + staple + O）	代码-物理	深挖 : 可乘性组合 $O = M^{tx;tx} + M^{ty;ty} - 2M^{xy;xy}$ 为本库独特之处
C3 Z_R 自重整化	算法	深挖 : arXiv:2510.17758 参数化全局拟合
C4 混合方案 λ 外推	算法	深挖 : 短距比值 + 长距 Z_R 拼接
C5 NLO 匹配核 g_0..g_3 与 Z_ij	代码-物理	深挖 : 单圈胶子匹配核 + 矩阵反演
C6 连续极限外推	算法	浅析: 仅拟合入口，未系统化
C7 torch 后端 + h5 IO + MPI 并行	工程	深挖 : 三后端统一适配层, 实测 2.7–4.8x 加速
C8 蒸馏管线 run_pipeline	工程	浅析: 调度层，逻辑照抄 docker
C9 数据分析链（analysis/）	工程	浅析: meff/ratio/disconnected 编排
排除 cpp/ 占位	—	排除 : 仅占位未实现
排除 refer/ 参考库	—	排除 : 第三方参考，逻辑照抄不 import

表 1: 核心部分候选三态分配（穷尽枚举，C1..C9 + 2 排除项）

2 项目性质判定

判定依据（实测占比）：仓库主体是胶子 TMD 计算的物理代码（renorm/operator 含显式公式与物理注释），同时每个算法都对应明确物理量（规范场、场强、Wilson 线、矩阵元）。故判定为**代码-物理交叉向**。报告剖析框架：对每深挖部分先讲物理图像，再给算法原理与对象映射表（代码符号 ↔ 物理对象双向可查），最后评估独特性。

3 核心部分深度剖析

3.1 C1 Wilson flow 梯度流 (RK3)

目的与输入 以 Wilson flow (Luescher 2010) 对非微扰规范场做平滑涂抹, 提供梯度流重整化方案所需的流规范场 V_τ 。输入: 规范场 U , 流时间 τ (物理约定 $\tau = 3a^2$, NieMiera 2025), 步长 $\varepsilon = 0.01$ 。

算法原理 流方程右端 $Z(V) = P_{ah}[\Omega \cdot V^\dagger]$ (无迹反厄米 $\in su(3)$), RK3 单步 $V(t+\varepsilon) = \exp(\sum c_i Z_i) \cdot V(t)$ (`pyqcd/renorm/_gradient_flow.py:111-135`)。

对象映射

代码符号	物理对象	含义
<code>U</code>	规范场链接	$g_\mu(x) \in SU(3)$, 张量 $(Nt, Nz, Ny, Nx, 4, 3, 3)$
<code>staple_6</code>	六-link staple	流方程生成元 Ω 的几何来源
<code>flow_derivative</code>	$Z(V)$	反厄米投影 $P_{ah}[M] = (M - M^\dagger)/2 - \text{Tr}(\cdots)/2N$
<code>wilson_flow_step</code>	RK3 步	$\varepsilon \leq 0.05$ 保 $O(\varepsilon^3)$ 精度
<code>wilson_flow</code>	演化到 τ	返回 V_τ 供后续算符构造

表 2: C1 对象映射表

正确性与独特性 正确性由 RK3 截断误差 $O(\varepsilon^3)$ 保证; 独特性在于统一后端 (numpy/cupy/torch) 实现且经 torus 一致性验证 (AGENTS.md:64)。

小结 C1 是非微扰重整化的基石, 实现标准但工程可移植性强。

3.2 C2 胶子 TMD 算符 (Clover F + staple + O 组合)

目的与输入 构造横向极化投影的胶子 TMD 算符矩阵元 $O(z, b_\perp)$ 。输入: 流规范场 V_τ 、纵向位移 z 、横向位移 $b_\perp$ 、方向 z_dir, b_dir 。

算法原理 (1) Clover 场强 $F_{\mu\nu} = (-i/8) \sum_k (P_k - P_k^\dagger)$ 四叶平均 (`pyqcd/operator/_gluon_operator.py:63-115`) ; (2) staple Wilson 线 $W_{st}(z, b_\perp)$ 连接分离点; (3) 非定域矩阵元 $M^{\mu\lambda;\nu\rho}(z, b_\perp) = \sum_x \text{Tr}[F^{\mu\lambda}(x + z\hat{z} + b_\perp) W_{st} F^{\nu\rho}(x) W_{st}^\dagger]$ (`pyqcd/renorm/_tmd.py:123-149`) ; (4) 可乘性重整化组合

```

1 def gluon_tmd_operator(U, z, b_perp, z_dir=2, b_dir=0, L=None):
2     """可乘性重整化组合  $O(z, b_\perp) = M^{\{tx;tx\}} + M^{\{ty;ty\}} - 2M^{\{xy;xy\}}$  (Eq.:0_mult_tmd) 。
3
4     Returns:
5         逐格点 0 值, 形状  $(Nt, Nz, Ny, Nx)$ 。
6     """
7     M_ttxt = M_mu_lambda_nu_rho(U, 0, 1, 0, 1, z, b_perp, z_dir, b_dir, L)
8     M_tyty = M_mu_lambda_nu_rho(U, 0, 2, 0, 2, z, b_perp, z_dir, b_dir, L)

```

```

9   M_xyxy = M_mu_lambda_nu_rho(U, 1, 2, 1, 2, z, b_perp, z_dir, b_dir, L)
10  return M_txtx + M_tyty - 2.0 * M_xyxy

```

物理图像 $O = M^{tx;tx} + M^{ty;ty} - 2M^{xy;xy}$ 即横向极化投影的胶子 TMD 算符 (Eq.:O_mult_tmd); 三项虚部在色迹 + 空间求和后归零, 故 O 为实数 (pyqcd/renorm/_tmd.py:164-181)。

对象映射

代码符号	物理对象	含义
plaquette_clover	$F_{\mu\nu}$	Clover 四叶平均场强
staple_wilson_line	W_{st}	staple Wilson 线 (非局域连接)
M_mu_lambda_nu_rho	$M^{\mu\lambda;\nu\rho}$	算符色迹 (未空间求和)
gluon_tmd_operator	$O(z, b_\perp)$	可乘性组合 (TMD 投影)

表 3: C2 对象映射表

正确性与独特性 正确性: 色迹维序 t, z, y, x 、roll 轴序严格对应位移; 独特性: 该 O 组合是梯度流胶子 TMD 方案的核心, 区别于朴素准 PDF (仅 $M^{tx;tx}$ 类)。

小结 C2 是本库最具物理独特性的部分, 直接决定可观测的胶子 TMD 信号。

3.3 C3 Z_R 自重整化

目的与输入 以自重整化方案 (arXiv:2510.17758 Eq.3-8) 参数化并全局拟合 $Z_R(z, a, \mu)$ 。输入: 各 z, a, μ 下的裸/重整化比值数据集、拟合初值。

算法原理 th_ZR 给出理论参数化, cost_function_all 汇总多数据集 χ^2 , fit_ZR 用 iminuit 全局拟合 (pyqcd/renorm/_zr.py:84-135, 155)。

对象映射 th_ZR $\leftrightarrow Z_R(z, a, \mu; k, d, m_0, m_2, \lambda_{QCD})$ 参数化; fit_ZR $\leftrightarrow$ 最小 χ^2 拟合。

独特性 复用 zengch 约定但自包含实现, 支持多系综联合约束。

3.4 C4 混合方案 λ 外推

目的与输入 短距比值 + 长距 Z_R 拼接, 对辅助参数 λ 外推到物理点。输入: $h_R(z, P_z)$ 、 λ 网格、拟合初值。

算法原理 hR_lambda_fit_form 定义外推形式, fit_hR_lambda 做 χ^2 拟合, hR_lambda/hR_x 给出外推后的 h_R (pyqcd/renorm/_hybrid.py:59-119)。

独特性 将短距可算部分与长距 Z_R 部分以 λ 外推连接，规避算符混合。

3.5 C5 NLO 匹配核 $g_{0..g_3}$ 与 Z_{ij}

目的与输入 由准 TMD 到光锥 PDF 的 NLO 匹配: $h_R^{PDF} = Z^{-1} \cdot h_R$ 。输入: x 网格、动量 P_z 、匹配标度 $\mu = 2 \text{ GeV}$ 、截断 $\lambda_s = 0.3 \text{ fm}$ 。

算法原理 按 $\xi = x/y$ 分区计算匹配核 $g_{0..g_3}$ (`pyqcd/renorm/_matching.py:25-61`), Z_{ij} 矩阵结构 (zengch 约定 $A_s = \alpha_s/4\pi$)

$$Z_{ij} = \delta_{ij} + \frac{\alpha_s C_A}{2\pi} dx \frac{x}{|x|} \times \left[\frac{g_{ij}}{y_j} - \delta_{ij} y_j \sum_k \frac{g_{ik}}{y_k^2} \right], \quad (1)$$

反演得 $h_R^{PDF} = Z_{ij}^{-1} h_R(y)$ (`pyqcd/renorm/_matching.py:94-105`; `pyqcd/renorm/_tmdextract.py:116,155`)。

对象映射 $g_i \leftrightarrow$ 单圈胶子匹配核分区贡献; $Z_{ij} \leftrightarrow$ 重整化群核 ($\delta + \alpha_s C_A/2\pi$ 核); $h_R^{PDF} \leftrightarrow$ 光锥胶子 PDF。

独特性 复用 zengch 约定但直接实现 Z_{ij} 矩阵反演，含快度演化与软函数 (`_tmdextract`)。

3.6 C6 连续极限外推 (浅析)

仅给出拟合入口 `fit_hR_PDF_extrap` (`pyqcd/renorm/_extrapolate.py:33-55`)，未做 $a/P_z/m_\pi/L$ 联合外推的系统误差量化，标注未验证。

3.7 C7 torch 后端 + h5 IO + MPI 并行 (深挖)

目的与输入 以 torch 后端替换 numpy/cupy 做 GPU/CPU 统一计算；产物统一 h5 读写；MPI 按 $N \cdot a = n \cdot b$ 调度 `step×conf` 元任务。

算法原理 `set_backend('torch')` 切换；torch.Tensor 补丁 `transpose/astype/repeat` 对齐 numpy 语义 (`pyqcd/tools/_torch_backend.py:222-254`)；`save_tensor_h5/load_tensor_h5` 通用 IO；`plan_parallel` 给出进程数 N 、批次 X 、每卡进程 Y 。

正确性与独特性 实测 torch CPU 梯度流 2.7–4.8x (逐位一致 $\max|d| \sim 10^{-15}$)；rank 0 独占分析作图 (不并行)。独特性：三后端零改算法代码、h5 可移植。

小结 C7 是工程竞争力的核心，使规模化实战 (test7: 100 组态) 可行。

3.8 C8 蒸馏管线 run_pipeline (浅析)

run_pipeline (pyqcd/pipeline/_steps.py:1418) 调度 9 步
(env→vertex→2pt→ope→3pt→4pt→analysis→plots→report) , 逻辑照抄 docker-v20260805, 自包含实现。

3.9 C9 数据分析链 (浅析)

analysis/ 实现 meff/ratio_fit/disconnected/色散拟合与三方向 ana_3dir, 复用 _disconnected 的 sem/resample/cov_mat 基元, 纯工程编排。

4 独特性评估

核心部分	独特性	对照基准
C2 算符 O 组合	高 (梯度流胶子 TMD 投影)	朴素准 PDF 仅单指标
C5 匹配核 Z_ij	中 (自包含矩阵反演)	zengch 公式移植
C7 三后端骨架	高 (零改算法 +GPU 加速)	单 numpy 实现
C1 梯度流	低 (标准 RK3)	Luescher 2010
C3/C4 Z_R/混合	中 (多系综联合)	文献参数化

表 4: 独特性评估 (高/中/低)

5 关键片段与公式

5.1 Wilson flow RK3 单步

```
1 def wilson_flow_step(U, eps):
2     """单步 RK3 (Luescher 2010 Eq.(3.8)) : V(t+ε) = exp(Σ c_i Z_i) · V(t)。"""
3     cp = get_backend()
4     W0 = U
5     Z0 = flow_derivative(W0)
6     W1 = su3_exp(0.25 * eps * Z0) @ W0
7     Z1 = flow_derivative(W1)
8     W2 = su3_exp((8.0 / 9.0) * eps * Z1 - (17.0 / 36.0) * eps * Z0) @ W1
9     Z2 = flow_derivative(W2)
10    return su3_exp((3.0 / 4.0) * eps * Z2 - (8.0 / 9.0) * eps * Z1
11                  + (17.0 / 36.0) * eps * Z0) @ W2
```

5.2 Clover 场强 (节选)

```
1 def plaquette_clover(g, mu: int, nu: int):
2     """F_{μν} = -i/8 Σ_k (P_k - P_k†), 四叶 Clover 平均 (donghx) 。
3
```

```

4  Args:
5      g: gauge, 形状 (Nt,Nz,Ny,Nx,4,3,3), CPU(ndarray) 或 GPU(cupy)。
6      mu, nu: Lorentz 指标 (0=t, 1=z, 2=y, 3=x), mu != nu。
7  Returns:
8      F_{\mu\nu}, 形状 (Nt,Nz,Ny,Nx,3,3)。
9      """
10     cp = get_backend()
11     m = cp.matmul
12     a_mu = 3 - mu    # 空间轴
13     a_nu = 3 - nu
14
15     g_lu = cp.roll(g, 1, axis=a_mu)
16     g_rd = cp.roll(g, 1, axis=a_nu)
17     g_ld = cp.roll(g_lu, 1, axis=a_nu)
18
19     tr = (0, 1, 2, 3, 5, 4) # 共轭转置 (色指标 Hermitian 共轭)
20
21     # P1 = P_{\mu\nu} (einsum→cuBLAS matmul 等价改写, 逐位一致)
22     p1 = m(g[..., mu, :, :],
23            cp.roll(g, -1, axis=a_mu)[..., nu, :, :])
24     p1 = m(p1, cp.roll(g, -1, axis=a_nu)[..., mu, :, :].conj()
25            .transpose(*tr))
26     p1 = m(p1, g[..., nu, :, :].conj().transpose(*tr))
27
28     # P2 = P_{\nu,-\mu}
29     p2 = m(cp.roll(g_lu, -1, axis=a_mu)[..., nu, :, :],
30            cp.roll(g_lu, -1, axis=a_nu)[..., mu, :, :].conj()
31            .transpose(*tr))
32     p2 = m(p2, g_lu[..., nu, :, :].conj().transpose(*tr))
33     p2 = m(p2, g_lu[..., mu, :, :])
34
35     # P3 = P_{-\mu,-\nu}
36     p3 = m(cp.roll(g_ld, -1, axis=a_nu)[..., mu, :, :].conj()
37            .transpose(*tr),
38            g_ld[..., nu, :, :].conj().transpose(*tr))

```

5.3 NLO 匹配核 $g_{0..g_3}$

```

1  def _matching_kernels(cxi, y_matr, Pz_GeV, mu_, lambda_s):
2      """按  $\xi=x/y$  分区计算匹配核  $g_{xy}$  ( $g_0$  + 对应分区的  $g_1/g_2/g_3$ )。"""
3      def g_1(xi_, y_):
4          return (-2.0 * (1.0 - xi_ + xi_ ** 2.0) ** 2.0 / (1.0 - xi_)
5                  * np.log(xi_ / (xi_ - 1.0))
6                  - (11.0 - 28.0 * xi_ + 18.0 * xi_ ** 2.0 - 12.0 * xi_ ** 3.0)
7                  / (6.0 * (1.0 - xi_)))
8
9      def g_2(xi_, y_):
10         res = (2.0 * (1.0 - xi_ + xi_ ** 2.0) ** 2.0 / (1.0 - xi_)
11               * (-np.log(mu_ ** 2.0 / (4.0 * y_ ** 2.0 * Pz_GeV ** 2.0))
12                 + np.log(xi_ * (1.0 - xi_))))
13         res -= ((15.0 - 56.0 * xi_ + 102.0 * xi_ ** 2.0
14                 - 96.0 * xi_ ** 3.0 + 48.0 * xi_ ** 4.0)
15                 / (6.0 * (1.0 - xi_)))
16         return res

```

```

17
18 def g_3(xi_, y_):
19     return (2.0 * (1.0 - xi_ + xi_ ** 2.0) ** 2.0 / (1.0 - xi_)
20             * np.log(xi_ / (xi_ - 1.0))
21             + (11.0 - 28.0 * xi_ + 18.0 * xi_ ** 2.0 - 12.0 * xi_ ** 3.0)
22             / (6.0 * (1.0 - xi_)))
23
24 def g_0(xi_, y_):
25     return (5.0 / 6.0 * (-1.0 / np.abs(1.0 - xi_)
26                        + 2.0 * Si(((1.0 - xi_) * np.abs(y_) * lambda_s))
27                        / (pi * (1.0 - xi_))))
28
29 mask_1 = (cxi < 0)
30 mask_2 = (0 < cxi) & (cxi < 1)
31 mask_3 = (cxi > 1)
32
33 g_xy = np.zeros_like(cxi)
34 g_xy[mask_1] = -g_0(cxi[mask_1], y_matr[mask_1]) - g_1(cxi[mask_1], y_matr[mask_1])
35 g_xy[mask_2] = g_0(cxi[mask_2], y_matr[mask_2]) + g_2(cxi[mask_2], y_matr[mask_2])
36 g_xy[mask_3] = g_0(cxi[mask_3], y_matr[mask_3]) + g_3(cxi[mask_3], y_matr[mask_3])
37 return g_xy

```

5.4 torch Tensor 补丁

```

1 def _patch_tensor():
2     if getattr(torch.Tensor, '_pyqcd_patched', False):
3         return
4
5     def transpose(self, *axes):
6         """numpy-style transpose: ``x.transpose()`` / ``x.transpose(axes)`` /
7         ``x.transpose(a, b, ...)`` — arbitrary axis permutation."""
8         if len(axes) == 0:
9             return torch.permute(self, list(range(self.ndim))[:-1])
10        if len(axes) == 1 and isinstance(axes[0], (tuple, list)):
11            axes = tuple(axes[0])
12        if len(axes) == 2:
13            return torch.transpose(self, axes[0], axes[1])
14        return torch.permute(self, tuple(axes))
15
16    def astype(self, dtype):
17        """numpy-style dtype cast (accepts np dtype / str / torch dtype)."""
18        return self.to(_resolve_dtype(dtype))
19
20    def get(self):
21        """cupy-compat: return the tensor as a numpy array on CPU."""
22        return self.detach().cpu().numpy()
23
24    def repeat(self, repeats, axis=None):
25        """numpy-style repeat with axis support (element-wise interleave);
26        without ``axis`` falls back to torch block repeat semantics."""
27        if axis is not None:
28            return torch.repeat_interleave(self, repeats, dim=axis)
29        if isinstance(repeats, (tuple, list)):
30            return torch.repeat(self, *repeats)

```


31

32

33

```
        return torch.repeat(self, repeats)

    torch.Tensor.transpose = transpose
```

6 参考源清单

参考源	类型	内容/作用
pyqcd/renorm/_gradient_flow.py:111-135	仓库	flow_derivative / wilson_flow_step RK3
pyqcd/renorm/_gradient_flow.py:163-173	仓库	flow_action_density E(t)
pyqcd/operator/_gluon_ope.py:63-115	仓库	plaquette_clover 四叶平均
pyqcd/renorm/_tmd.py:123-161	仓库	M 矩阵 + O 组合
pyqcd/renorm/_tmd.py:203-213	仓库	self_renormalized_ratio
pyqcd/renorm/_zr.py:84-135	仓库	th_ZR / fit_ZR
pyqcd/renorm/_hybrid.py:59-119	仓库	hR_lambda 混合外推
pyqcd/renorm/_matching.py:25-105	仓库	g_0..g_3 与 Z_ij 反演
pyqcd/renorm/_tmdextract.py:116	仓库	Z_ij 数值结构
pyqcd/renorm/_extrapolate.py:33-55	仓库	外推拟合入口
pyqcd/tools/_torch_backend.py:222-254	仓库	torch 补丁
pyqcd/pipeline/_steps.py:1418	仓库	run_pipeline 9 步
AGENTS.md:64	仓库	torch 加速实测 2.7-4.8x
refer/	参考	zengch/donghx/git-rep 等（逻辑照抄不 import）

7 结论与局限

结论

最强竞争力: (1) C2 梯度流胶子 TMD 算符 O 组合 (物理独特); (2) C5 NLO 匹配核 Z_ij 自包含反演; (3) C7 三后端可移植工程骨架 (工程独特)。三者共同支撑「梯度流重整化胶子 TMD-PDF」端到端自包含、可复现、可扩展的实现。核心部分穷尽枚举 9 候选 + 2 排除, 三态闭合。

物理图像

物理图像主线: 裸矩阵元 → 梯度流涂抹 → 可乘性 O 算符 → Z_R/混合重整化 → NLO 匹配 → 连续极限外推, 对应代码链 C1→C2→C3/C4→C5→C6, 对象映射双向可查。

局限与风险

未覆盖/未验证：(1) C6 连续极限外推仅入口，未量化 $a/P_z/m_\pi/L$ 联合系统误差，标注未验证；(2) cpp 后端占位未实现；(3) refer 内部库仅参考未深剖；(4) 部分运行时数值（显存/耗时）引自 AGENTS.md 实测，未独立重测。